

## PRE-CLASS READING 1 OF 5

# How each kind of AI breaks — and why the control that catches one catches none of the others

*Instructor-assembled brief.* This is the one reading in the Week-6 set that the OBTL+ mandatory list does not name, and it is here for a specific reason: Capsule 1 asks you to match a failure mode to an AI type and then to design the guardrail that catches it, and not one of the other four readings in this set describes a single concrete failure. They describe obligations. This brief describes breakage. Where it draws on a published source — NIST's Generative AI Profile, the 2026 International AI Safety Report, documented incidents — the source is named and linked at the end.

## ASSIGNED READING

**This brief is the assigned reading.** There is no external source to read first. Two optional companions if you want the primary material: **NIST AI 600-1**, the Generative AI Profile (July 2024), for its twelve named risk categories — skim the risk names, not the forty-odd pages of suggested actions; and the **International AI Safety Report 2026** (3 February 2026), chaired by Yoshua Bengio, for the current empirical picture. Both are free. Neither is examinable.

» this brief ~22 min • optional sources ~25 min

**Note on currency.** Everything in the failure taxonomy below is structural and will not date. The *incidents* used as illustrations will: they are drawn from 2023–2026 and newer and worse ones will exist by the time you read this. The point of an incident here is never “this happened”, it is “this is the shape the failure takes”, and the shape is stable. Where a figure comes from a survey or a vendor, it is marked as such.

## KEY TAKEAWAY

There is no such thing as “AI failure”. There are four kinds of AI in this course and they break in four incompatible ways, and the control that catches one is silent on the others. **A discriminative classifier fails silently, statistically, and on a subgroup** — it returns a confident label that is wrong for a population you were not looking at, and nothing in the output says so. **A large language model fails fluently and per-instance** — it produces a well-formed, plausible, confidently-worded answer that is false, and because fluency and correctness are generated by the same mechanism the output carries no distress signal at all. **An agent fails by acting** — the error is not a wrong answer you can discard but a sequence of steps taken in the world, so the failure has a blast radius and sometimes cannot be undone. **An embodied system fails physically** — irreversibly, at the speed of hardware, and the review step you were relying on does not exist because there is no time for one. Two properties sort these cleanly and are worth more than any list: **can you tell from the output alone that it failed**, and **can you undo it**. Classifier failures are invisible and reversible. Language-model failures are invisible and usually reversible. Agent failures are visible after the fact and often irreversible. Embodied failures are visible and never reversible. Your guardrail budget should follow that grid, not the alphabet.

## WHY IT MATTERS FOR THIS COURSE

This is the reading that closes the loop the course opened. Week 1 taught you that AI types differ in what they can do; this brief is the same claim viewed from the other side — **they differ in how they break, and the difference is not cosmetic**. Capsule 1 opens with a poll asking you to match a failure mode to an AI type, and it is deliberately harder than it looks. Capsule 2's build asks you to attack your own Week-3 or Week-4 system and then harden it, and you cannot design a guardrail without a theory of what you are guarding against. And A2 — due this week — requires a responsible-deployment guardrail checklist with at least ten test cases. Ten test cases against no taxonomy is ten variations on the same test. **The taxonomy is what makes the tenth test different from the first.**

**Feeds Class 6: C1 (failure modes per modality • the eval/observability/guardrail/governance stack) • C2-B (what you are attacking, and what a fix actually fixes) • A2 (the guardrail checklist).**

## MAIN POINTS

### The four failure modes, and the property that separates them

Start from a single observation that most deployment conversations skip: **the reason these four break differently is that they are asked for different things**. A classifier is asked for a label, and a label is a compressed claim about a population. A language model is asked for text, and text is judged by whether it reads correctly. An agent is asked for an outcome, and it gets there by acting. An embodied system is asked for a movement, and a movement is a change to the physical world. Each of those four contracts has a different relationship to error, and that relationship — not the algorithm — is what determines the failure mode.

- **Discriminative ML — the silent statistical failure.** The model returns a label and a confidence score, and the confidence score is about the *model's fit to its training distribution*, not about whether that distribution still describes the world. When the world moves — new customers, a new product, a new season, a competitor's promotion — the model does not notice. It keeps returning confident labels, and the accuracy degrades for

months without a single error message. The failure is **statistical**, so it is invisible in any individual case: no single prediction looks wrong, and the damage is only visible in aggregate, and only if you are aggregating. And it concentrates: a model that is 94% accurate overall can be 71% accurate on a subgroup that is 6% of the population, and the headline number will never show you that.

- **Large language models — the fluent per-instance failure.** NIST calls it **confabulation** and the word is better than “hallucination”, because hallucination implies a sensory malfunction and confabulation is what the system does when it is working exactly as designed. The mechanism matters and is worth stating plainly: a language model is trained to produce text that is *likely*, and a fluent false statement and a fluent true statement are both likely. **Correctness is not a variable the model optimises, so it is not a variable the output reports on.** This is the single most consequential fact in the entire course for anyone about to deploy one, and it has a direct corollary that people resist: you cannot detect a confabulation by reading it. It does not hedge, it does not slow down, it does not produce a lower confidence. It produces the same shape of sentence as the truth, because the same mechanism produced both.
- **Agents — the failure that has already happened.** Week 4 gave you the arithmetic: a 95% per-step success rate over twenty steps is  $0.95^{20} \approx 36\%$ , and a ReAct loop’s cost grows with the square of the turns. This brief adds the other half. A wrong *answer* is a thing you can throw away. A wrong *action* is a thing that has already been done — an email sent, a record updated, a refund issued, a file deleted. And because agents chain, the second step is conditioned on the first: **an early error is not an error, it is a premise**, and the agent will reason competently from it for another fifteen steps. The failure mode is not “the agent was wrong”, it is “the agent was wrong at step three and confidently correct about everything downstream of being wrong”.
- **Embodied and physical AI — the failure with no review step.** Every control discussed in this brief so far assumes a gap between the decision and its consequence, in which something can inspect the decision. A vehicle, a robot arm, a surgical instrument or a warehouse mover closes that gap to nothing. There is no human-in-the-loop at 60 km/h. So the entire safety burden shifts *before* deployment — into simulation, formal constraint, physical envelope limits, redundant sensing and hard mechanical stops — and the operational-monitoring layer that carries most of the weight for the other three types can only tell you what went wrong afterwards. This is why the embodied-AI safety literature looks so different from the LLM safety literature: it is not a more cautious version of the same discipline, it is a different one.

### The grid — detectability against reversibility, and what it tells you to spend on

Put the two properties on axes and the four types separate cleanly. **Detectability from the output alone** runs low to high; **reversibility** runs low to high. The classifier sits low-detectability, high-reversibility: you cannot see the failure but you can undo the decision, so **your money goes on monitoring** — drift detection, subgroup slicing, a held-out audit set scored on a schedule. The language model sits low-detectability, mostly-reversible: **your money goes on verification** — grounding the answer in a retrievable source, structural validation, a second check that is cheaper than a human. The agent sits higher-detectability (you can see what it did) but low-reversibility: **your money goes on gates** — confirmation before any irreversible or externally-visible action, step and cost caps, tool permissions scoped to the task. The embodied system sits high-detectability, zero-reversibility: **your money goes before deployment**, into simulation and physical constraint, because there is no after.

**The transferable rule: spend on detection where the failure is invisible, and spend on prevention where the failure is permanent. A team that puts its entire budget into one of those two is protected against half its failure modes and does not know which half.**

**And the diagnostic that follows from it, which is the thing to take into A2.** For any component of your system, ask the two questions in order. *If this is wrong, will anything tell me?* If no, you have a monitoring problem and no amount of guardrail-at-the-boundary will help, because the boundary sees a well-formed output. *If this is wrong, can I undo it?* If no, you have a gating problem and no amount of monitoring will help, because by the time the monitor fires the thing has happened. Most real systems have components in more than one quadrant — a retrieval step (invisible, reversible), a generation step (invisible, reversible), and one tool call that writes to a system of record (visible, irreversible) — and **the mistake almost every first design makes is applying one control uniformly across all three.**

### The worked example — why “have a human check it” is usually not a plan

This is the arithmetic to bring to class, because it is where good intentions meet a budget. Take a support-triage assistant handling **50,000 tickets a month** — the same volume Week 5 costed — and suppose the model is **95% correct**. That is 2,500 wrong answers a month, and you have decided a wrong answer is unacceptable, so you will have a human check the output. Work it through.

1. **Review everything.** In principle you catch all 2,500 — in practice you catch whatever fraction a reviewer actually detects, which Q5 argues is well under one (a reviewer who has approved four hundred correct outputs is not really reading the four hundred and first). You have also paid a human to read 50,000 answers, which is the workload you deployed the system to remove. Net saving: the difference between reading an answer and

writing one — real, but a fraction of what the business case promised, and you should find out now rather than in month four.

2. **Review a random 10%.** You have paid a tenth as much and you catch 250 of the 2,500 — and note this is not like-for-like with the next option, which reviews a *targeted* 5%. The other 2,250 ship. Random sampling is excellent for *estimating* your error rate and useless for *preventing* individual errors, and these two purposes are confused constantly. Sampling is measurement, not a guardrail.
3. **Review selectively — the only version that works, and it needs a signal.** To review the 5% most likely to be wrong you need something that correlates with wrongness. Here is the problem this brief exists to make unavoidable: **the model's own fluency does not correlate with its correctness, and neither, in general, does its stated confidence.** A confabulated citation is delivered in exactly the register of a real one. So a selective-review strategy has to *manufacture* a signal, and the ones that work all cost something.

**Three signals that are actually available, with their real costs. Self-consistency:** sample the same question three times at non-zero temperature and flag disagreement. It works because a fact the model knows is stable across samples while a confabulation is often reconstructed differently each time — but it triples your token cost on every request, and it is silent when the model is *consistently* wrong, which is exactly what happens when the error comes from the training data rather than from sampling. **Grounded verification:** require every claim to be traceable to a retrieved passage and check the trace. It is the strongest of the three and it is the reason Week 2's RAG build matters here, but it only covers claims that *have* a source, and it converts your problem into a retrieval-quality problem — which is a real problem with its own failure mode, LLM09:2026 in the OWASP list. **Structural validation:** demand JSON against a schema, check enums, check ranges, check that cited document IDs exist. It is nearly free, it runs in your own code, and it catches a genuinely useful class of error — but it validates *form*, and a well-formed lie passes every time.

**The conclusion, and it is the one to bring to A2: “a human will check it” is a cost, not a control, until you can say which outputs the human sees. The design work is not adding review — it is manufacturing the signal that makes review affordable. Cost per successful task (Week 5) is the right frame: the review layer is a term in that fraction, and a review strategy that reads everything has quietly doubled your denominator.**

#### The four operational layers, and what each one can and cannot see

Responsible deployment in practice is four layers, and they are not alternatives. Each catches a class of problem the others are blind to, and teams routinely build one and believe they are covered.

- **Evaluation — before deployment, on cases you chose.** Your Week-3 ten-case harness, extended. Its power is that it is repeatable, so it turns “is the new prompt better?” into a question with an answer. Its blind spot is total and worth saying out loud: **an eval set can only contain failures you imagined.** This is the thread from Week 3 — you cannot test for a failure you did not think of — and it is why an eval suite is necessary and never sufficient.
- **Observability — after deployment, on cases the world chose.** Logging inputs, outputs, tool calls, latencies, costs and, critically, whatever ground truth eventually arrives (the ticket that got reopened, the refund that got reversed, the recommendation nobody clicked). This is the only layer that sees the failures your eval set lacked the imagination for. Its blind spot: it tells you afterwards, and for irreversible actions afterwards is too late.
- **Guardrails — at runtime, in your own code.** Input filters, output schemas, tool permission scopes, step caps, spend caps, confirmation gates, refusal rules. The distinction Week 4 established governs the whole layer: **guarantees belong in code, judgement belongs in the prompt.** A cap you implement in Python holds when the model is being manipulated; an instruction in the system prompt asking the model to be careful is a request, and the whole of the OWASP list is about what happens to requests. Blind spot: guardrails enforce what you specified, and they are silent on everything else.
- **Governance — around all of it, on a slower clock.** Who signed off, who is accountable when it fails, what triggers a rollback, who may change the prompt, how an incident is reported and to whom, what is retained and for how long. This is the layer the five assigned Week-6 documents are almost entirely about, and it is the one engineers skip and the one regulators check. Its blind spot is that it governs process and cannot see behaviour: **a fully compliant governance regime around a badly-behaved model is exactly what a lot of AI governance currently looks like.**

**The DBS thread, closed.** Week 2 gave you DBS's P-U-R-E test — that a deployed system be **Purposeful, Unsurprising, Respectful and Explainable** — and at the time it read as a values statement. Read it now against the four layers and it is a specification. *Purposeful* is a governance question (does this use case have a stated, approved purpose?). *Unsurprising* is an observability question (is behaviour in production consistent with behaviour in test?). *Respectful* is an evaluation question, and specifically a subgroup-slicing one. *Explainable* is a guardrail-and-architecture question, and it is the one generative systems make hardest. A bank arrived at that list before large language models existed, and it has aged better than most things written since.

## WHERE TO BE SCEPTICAL

Four things this brief has told you that deserve pushing back on, including one where the honest answer is that the field does not agree.

1. **“Fluency does not correlate with correctness” is a strong claim, and it is not quite universal.** There are real, measurable signals in a model’s internals — token-level log-probabilities, entropy over the output distribution, and a research literature on probing internal activations for a truthfulness signal — and some of them beat chance. What is true is the practical version: **the signal is weak, unreliable across domains, and frequently unavailable**, because most production deployments call a hosted API that does not return log-probabilities (and OWASP recommends gating them precisely because they leak information). Treat “no free signal” as a sound default for system design and not as a claim about what is possible in principle.
2. **The four-type taxonomy is a teaching device, and real systems are hybrids.** A production assistant is a retrieval model, a classifier for routing, a language model for generation and a set of tools, and the failures interact in ways no single-type analysis predicts: a drifting router sends more traffic to the wrong prompt, which confabulates more, which the agent then acts on. Use the taxonomy to analyse **components**, not products. The system-level question is a different and harder one.
3. **Incident counts are not risk measures, and the number you are quoted is a reporting artefact.** Every published tally of AI incidents measures what somebody chose to report to a voluntary database. Sectors with mandatory reporting look dangerous; sectors with none look safe. This is the same three-question test Week 5 applied to survey statistics — **who was asked, what threshold counted, what was the denominator** — and it applies without modification to safety data. Be especially suspicious of any claim that incidents are rising, which is exactly what improving reporting also produces.
4. **Nobody has a good answer for evaluating agents, and you should know that before you build one.** Grading a classifier is easy — you have labels. Grading a generated answer is hard but tractable — a rubric, a stronger model as judge, a human sample. Grading a *trajectory* is genuinely unsolved: the agent reached the right outcome by a route that would fail next time, or the wrong outcome by sound reasoning from a bad tool response. The Week-4 doctrine was **grade the outcome, not the path**, which is right and is also an admission — the path is where the information about robustness lives, and we are choosing not to look at it because we do not know how.

## READING FOCUS QUESTIONS — bring your answers to class

There are no wrong answers here — these surface how you think, not what you recall. A clue sits under each question to give you a direction to push on; the fuller reasoning is in the section after.

- Take the system you built in Week 3 or Week 4 — or the one your A2 team is building. Put each of its components on the detectability × reversibility grid. Which quadrant is currently unguarded?  
*Clue.* Do it component by component, not system by system; the whole point of the grid is that one product usually occupies three quadrants at once. For each component ask the two questions in order — **if this is wrong, will anything tell me?**, then **if this is wrong, can I undo it?** — and notice that a “no” to the first sends you to monitoring while a “no” to the second sends you to a gate, and they are not substitutes. Expect to find that you have built controls for the quadrant you find easiest to reason about.
- Your model is 95% correct on 50,000 tasks a month and a wrong answer is unacceptable. You cannot afford to review everything. Design the review strategy — and state what signal makes it possible.  
*Clue.* Do the arithmetic before you design anything: 2,500 errors, and work out what each review policy actually catches. Then confront the hard part rather than assuming past it — selective review needs something that correlates with being wrong, and the model’s own confident tone is not it. Price the three signals in this brief (self-consistency triples your tokens; grounded verification needs retrievable sources; structural validation is nearly free and only sees form) and ask which of your errors each one would actually have caught. A defensible answer may well be **“we cannot review selectively, so we must change the task”** — narrowing scope until the errors become detectable is a legitimate design move, not a retreat.
- A discriminative classifier fails silently and a language model fails fluently. Both are invisible in the output. So why does the same monitoring strategy not work for both?  
*Clue.* Start from what each failure is a property of. One is a property of a **population** — it shows up as a distribution shifting away from what you trained on, so it is visible in aggregate over time and invisible in any single case, which tells you exactly what to aggregate and what to slice by. The other is a property of a **single response** — the model is not drifting, it produced a false sentence this once, and averaging a thousand of those tells you a rate rather than which one was wrong. Ask what ground truth arrives, and when, in each case; that is usually where the two strategies part company.
- The four operational layers each have a blind spot. Pick the layer your organisation is strongest at and name what it cannot see — then say who in your organisation would be responsible for that gap.  
*Clue.* The second half of the question is the real one. Each blind spot has an owner in principle and usually nobody in practice, and the gaps tend to fall precisely on the seams between functions — evaluation belongs to the

*builders, observability to platform or operations, governance to risk and legal, and guardrails are split across all three, because the function that decides what must be blocked is rarely the function that writes the code that blocks it. The failures that survive are the ones that need two of those groups to notice the same thing. Ask what would have to be true for the gap to be somebody's explicit job, and whether that person currently has the access to do it.*

- Embodied AI removes the review step entirely. Does that make it a special case — or does it just make visible a problem the other three types also have?

**Clue.** Resist the comfortable answer that physical systems are simply a separate discipline. Instead look for the review step in the other three and ask what it actually depends on: time between the decision and its consequence, and someone with the information and the authority to intervene in that window. Then ask how wide that window really is for an agent that sends an email in 200 milliseconds, or for a credit decision that a customer is told about instantly. Week 4's confirmation gates were an argument about exactly this, and it is worth asking whether they are a solution or a way of buying back a window that the architecture had removed.

## HOW TO THINK ABOUT THESE — read after you have had a go

*These are not model answers to be memorised — they are the line of reasoning we will pick up in class. Class time is short, so they are written to stand on their own.*

### Q1 — Almost every first design guards one quadrant and calls it done

The predictable result of doing this exercise honestly is discovering that your controls cluster. Teams with a strong engineering culture build guardrails — schemas, caps, retries — which protect the irreversible quadrant well and are entirely blind to the invisible one, because a schema-valid answer that is false passes every check you wrote. Teams with a strong data culture build monitoring, which catches drift beautifully and does nothing about the single tool call that wrote to a system of record. Neither team is careless; each has generalised from the failures it has seen before. Work an ordinary Week-4 build through it. The **retrieval step** is low-detectability and high-reversibility: if it returns the wrong passages nothing complains, and the fix costs a re-run — so it needs a retrieval eval and it needs logging of what was retrieved, which is the single most commonly skipped log in a RAG system and the one you will want first when something goes wrong. The **generation step** is the same quadrant and wants grounded verification. The **tool call that writes** is the different one — visible after the fact, and not undoable — and it wants a confirmation gate and a scoped permission, and it wants them **in your code**, because the Week-4 lesson holds: a guarantee written in the system prompt is a request. The specific thing to bring to class is which quadrant you found empty, because the reason it is empty is almost always that nobody owns it, and that is a more useful finding than the gap itself. **All of which compresses into six words worth carrying out of this course: gate the irreversible, monitor the invisible. They are two controls answering two different failures, and they are not substitutes — the reason your controls cluster is that most teams have only ever built one of them.**

### Q2 — The design work is manufacturing the signal, not adding the reviewer

The arithmetic first, because it settles the question faster than argument: 50,000 tasks at 95% is 2,500 errors, full review catches 2,500 and costs you the workload you were removing, random 10% catches 250 and lets 2,250 through. There is no third option that does not involve a signal. So the honest design question is which signal you can afford, and the answer depends on a property of your task that you should determine before writing any code: **do your outputs have sources?** If they do — the answer is drawn from documents, records, a catalogue, a policy — then grounded verification is available and it is much the strongest option, because it converts an unanswerable question (is this true?) into a mechanical one (does the retrieved passage say this?). If they do not — the model is summarising, drafting, reasoning, deciding — then you are left with self-consistency and structural validation, and you should be clear-eyed that structural validation catches format errors and nothing else. Self-consistency is the interesting middle case and it is worth understanding *why* it works, because it tells you when it will not: sampling three times at temperature explores the model's output distribution, and a fact that is well-represented in the weights is stable under that exploration while a confabulation is often re-improvised. Which means it detects **uncertainty**, not falsehood — and a model that has confidently learned something wrong will agree with itself three times out of three. That is not a small caveat; it is the entire class of errors that come from bad training data rather than from sampling noise. The fourth answer, and the one most likely to be right in a real deployment, is to **change the task until the errors become detectable**. A system that drafts a reply for a human to send has moved the irreversible step to a person and converted an unbounded correctness problem into a bounded review one. A system that returns three candidate answers with sources has made the human's job checking rather than authoring. **Both are worse than the demo and better than the incident, and both are the sort of trade-off Rubric 1 is asking you to make explicit.**

### Q3 — One failure is a property of a population, the other of a sentence

They are both invisible in the output and that is where the similarity ends. The classifier's failure is a **distributional** event: the input distribution moved and the decision boundary the model learned no longer sits where it should. That has three consequences that determine the monitoring strategy. It is gradual, so a threshold on a weekly aggregate will see it. It is detectable *without labels*, because you can watch the input distribution and the output

distribution shift even before any ground truth arrives — a sudden change in the proportion of predictions in each class is a real signal about a model nobody has told anything. And it is **concentrated**, which is why slicing matters more than the headline: overall accuracy is an average over subgroups and it is perfectly capable of holding steady while one subgroup collapses. So the strategy is drift detection on inputs, distribution monitoring on outputs, subgroup slicing on whatever labels eventually arrive, and a small held-out set scored on a schedule. The language model's failure is not distributional at all. Nothing has moved; the model is behaving exactly as it did in testing; it produced one false sentence in one response, and the next response about the same subject may be correct. Aggregating those gives you a confabulation *rate*, which is a genuinely useful number for deciding whether to deploy and completely useless for deciding whether **this** answer can be sent to a customer. And the ground truth is different in kind: for the classifier it arrives on its own, late but free — the loan defaulted or it did not, the ticket was reopened or it was not — whereas for the generated answer nothing arrives unless somebody complains, and non-complaint is not evidence of correctness. That asymmetry is why the two strategies diverge. The classifier wants **statistical monitoring over time**, and the labels turn up eventually. The language model wants **per-instance verification at the moment of generation**, because there is no later. **The practical trap worth naming: teams that have run classifiers for years bring their monitoring stack to a generative system, build excellent dashboards of latency, token volume, cost and thumbs-up rate, and are genuinely surprised when a confabulation reaches a customer — every dashboard was green, because none of them was measuring truth.**

#### Q4 — The blind spots fall on the seams between functions

The layer an organisation is strongest at is a good predictor of the incident it will have. A strong **evaluation** culture — common in teams that came from machine learning — produces confidence proportional to test coverage and is blind to the unimagined failure, which is by construction the one that happens; the specific danger is that a good eval suite feels like proof, and the better it is the more it feels like proof. A strong **observability** culture, common where there is a mature platform team, catches almost everything and catches it afterwards, which is adequate for a recommendation and inadequate for a payment. A strong **guardrail** culture enforces the enumerated and is silent on the rest, and its characteristic failure is a system that is very hard to break in the ways somebody thought of. A strong **governance** culture — common in banks, insurers, healthcare and anywhere with a regulator — produces documentation that is genuinely defensible and can coexist with a model nobody has evaluated on a subgroup, because governance sees process and not behaviour. The second half of the question is the one worth spending time on, because these gaps are not knowledge gaps. Almost everyone, asked directly, can name what their strongest layer misses. What is missing is ownership, and the pattern is consistent: the gaps sit on the seams. Evaluation belongs to the people who built it — unified ownership, and the one exception proves the rule, because in banking and insurance a regulator has already broken it up and made model validation an independent second line. Observability belongs to platform or operations. Governance belongs to risk and legal. **Guardrails are the awkward layer, and the most instructive one**, because they are the only layer whose ownership is split three ways by construction: risk, compliance or product decides *what must be blocked*, the builders write the code that blocks it — in the request path, in the same pull request as the feature — and in any organisation large enough to have shared filters, platform or trust-and-safety operates them. Three functions, one layer, and no single owner of the question that actually matters: **what did none of us think to enumerate?** That is not a side observation about org charts. It is the direct cause of the guardrail layer's characteristic failure — a system that is very hard to break in the ways somebody thought of — and it is the reason no regulator has yet done for guardrails what banking supervision did for model validation. A failure that needs two of those groups to notice the same signal is a failure that will not be noticed. Concretely, the most common uncovered gap in the organisations this course's cohort works in is the space between "the model was tested and passed" and "the model is being used for something nobody tested" — the use case drifted, quietly and reasonably, because the tool was useful and people found new things to do with it, and nothing in any of the four layers is watching for that. The NIST framework in Reading 3 and the IMDA frameworks in Reading 4 both have a name for the answer, and it is unglamorous: **someone accountable, named, with the authority to stop it. That is the part of a governance framework that does real work, and it is the part that is hardest to obtain.**

#### Q5 — The review step is a window, and the window has been closing everywhere

The right answer is the second one, and it is the most useful idea in this brief. Embodied AI is not a special discipline that happens to be harder; it is the limiting case of a property every deployed system has, made impossible to ignore because the consequence is physical. Look at what a review step actually requires and it is three things, none of them about robots: **time** between the decision and its consequence, **information** sufficient to judge the decision in that time, and **authority** to stop it. A vehicle has none of the three, which is why the safety work has to move entirely before deployment. But now audit the other three types honestly and the picture is less comfortable than it looks. An agent that sends an email has a window of milliseconds — Week 4's confirmation gate is precisely the recognition that the architecture had removed the window and someone had to buy it back, deliberately, at the cost of the autonomy that was the point. A credit decision delivered instantly to an applicant has a window of zero in practice even though the technology permits one, because the business chose instant. A classifier that silently routes tickets has, in principle, months of window and, in practice, none — because nobody is looking, and a window nobody occupies is not a window. That reframing does real work. It says that "human in the loop" is not a control

you can claim by putting a person on an org chart; it is a claim about time, information and authority, and it should be stated as such: *this human has this long, sees this, and can do this*. Written that way, a great many human-in-the-loop designs turn out to be a person who has four seconds, sees a confidence score, and is measured on throughput — which is not oversight, it is a rubber stamp with a salary, and it is worse than no oversight because it creates an accountable-looking record. It also explains why the automation-bias problem Singapore's agentic framework added in its May 2026 update is a genuine finding rather than a platitude: the reviewer who has approved four hundred correct outputs is not meaningfully reviewing the four hundred and first. So the general form, and the thing to carry into A2: **for every consequential action in your system, state the window. If the window is zero, you do not have a review step, whatever your architecture diagram says, and you must move the control before the action — a gate, a cap, a scope, a refusal — because after the action there is only observability, and observability is a record of what happened.**

## APPENDIX — VOCABULARY, EXPLAINED

The distinction that carries this reading: **a failure you cannot see, and a failure you cannot undo**. They are different problems with different budgets, and every term below belongs to one or the other. Read the two-layer entries as a pair — the **Plain**. line is what you would say in a meeting; the **Technically**. line is what you would need to defend it.

### Confabulation (and why it is a better word than hallucination)

- **Plain.** The model states something false in exactly the same confident tone it uses for something true.  
**Technically.** NIST AI 600-1 uses **confabulation** for the generation of confidently stated erroneous content. The term is better because hallucination implies malfunction: the system produced likely-sounding text, which is precisely what it was trained to do. **Correctness is not an objective the model optimises, so it is not a property the output signals.** Second of the twelve GenAI risks NIST names — the list is alphabetical, and CBRN information is first.

### Silent failure

- **Plain.** The model is wrong and nothing anywhere says so.  
**Technically.** A failure that produces no error, no exception and no lowered confidence — the system returns a well-formed answer of the expected type. Characteristic of both discriminative models under distribution shift and generative models confabulating. **The defining property of AI failure and the reason software monitoring practice does not transfer:** conventional systems fail loudly.

### Distribution shift (data drift and concept drift)

- **Plain.** The world stopped looking like the data you trained on.  
**Technically.** **Data drift:** the input distribution  $P(X)$  moves — new customers, new products, a new channel. **Concept drift:** the relationship  $P(Y|X)$  moves — the same input should now get a different label, as when fraud patterns adapt. Data drift is detectable without labels by monitoring input and output distributions; concept drift generally is not, and needs ground truth. Both degrade a model that reports no problem.

### Subgroup performance (and why the headline number hides it)

- **Plain.** Overall accuracy is an average, and averages hide the group it fails on.  
**Technically.** Aggregate accuracy is a weighted mean over subpopulations. A model at 94% overall can sit at 71% on a subgroup that is 6% of volume, with no movement in the headline. **Slicing** — evaluating separately by segment, geography, channel, language, device or any attribute you have — is the control, and it only finds what you thought to slice by. NIST names *Harmful Bias or Homogenization* as a distinct risk category for this reason.

### Compounding error

- **Plain.** A small per-step error rate becomes a large per-task one because the steps multiply.  
**Technically.** For  $n$  independent steps at per-step success  $p$ , task success is  $p^n$ :  $0.95^{20} \approx 36\%$ . Week 4's arithmetic. In an agent the steps are not independent — later steps are conditioned on earlier ones, so an early error becomes a **premise** the agent reasons competently from, which is worse than independence, not better.

### Excessive agency

- **Plain.** The system can do more than the job needs, so a small mistake reaches further than it should.  
**Technically.** OWASP's term (LLM03:2026, risen from LLM06:2025), decomposed into excessive **functionality**, excessive **permissions** and excessive **autonomy**. The canonical case: a mail-summarising assistant given a tool that can also *send*. Mitigation is scope, not accuracy — minimise tools, minimise each tool's functionality, execute in the user's own permission context, and require confirmation for consequential actions. See Reading 2.

**Evaluation and guardrails are not the same layer, and A2 marks them separately. An evaluation case asks did it get the job right? A guardrail case asks did it refuse, cap, or escalate when it should have? Evaluation measures behaviour you want and cannot enforce it; a guardrail enforces behaviour you require and measures nothing. A system with good evals and no guardrails can tell you it is wrong and do it anyway; a system with guardrails and no evals is reliably prevented from doing the things somebody already thought of.**

### Evaluation

- **Plain.** A test you set, on cases you chose, before anybody is affected.

**Technically.** Held-out sets, regression suites, task-specific rubrics, model-graded scoring — Week 3's ten-case harness is the small version of all of it. The defining limitation is structural rather than a matter of effort: **an evaluation can only contain failures somebody imagined**, so it is necessary and never sufficient, and the better it is the more it feels like proof. Ownership is unified — the people who build the system usually also decide what it is tested against. The exception is instructive: in banking and insurance, supervisors require **independent model validation** as a separate line, which is the only place the builders are not grading their own homework. **In A2 this is D4:** 30 to 50 cases written by your own team — 5 to 8 each — outcome-graded, which means grading what the agent concluded and not the path it took there, each case starting from a clean state. Between 2 and 10 of them are negative cases, where the correct outcome is the ask or the escalate rather than the act, and each gets three trials because negatives flip between runs and one trial cannot distinguish a real refusal from a lucky one. **The negatives exist because of the limitation above.** Class 4's eager agent scored 18 out of 18 on the six ordinary tasks and 18 out of 24 overall: *the dangerous agent is the one that is right about everything you tested.*

### Observability (as distinct from evaluation)

- **Plain.** Evaluation is a test you set; observability is a record of what actually happened.

**Technically.** Observability runs after deployment on cases the world selected — logging inputs, outputs, tool calls, retrieved context, latency, cost and any ground truth that later arrives. **The only layer that can see an unanticipated failure, and it sees it afterwards**, which is adequate for a recommendation and inadequate for a payment.

### Guardrail

- **Plain.** A rule enforced by your code, not requested of the model.

**Technically.** A deterministic runtime constraint: schema validation, allow-lists, tool permission scopes, step and recursion limits, spend caps, confirmation gates. The Week-4 doctrine — **guarantees belong in code, judgement belongs in the prompt** — is the whole of the distinction. A constraint written into the system prompt is a request, and Reading 2 is about what happens to requests. **Enforces the enumerated and is silent on the rest**, which is why the ownership question in Q4 matters: the function that decides what must be blocked is rarely the function that writes the code that blocks it. **In A2 this is D3, and it is two things.** (a) The code layer — a step cap, a budget ceiling, action de-duplication, and an explicit autonomy setting (suggest, confirm or act) with the gate placed in front of the irreversible step rather than in front of the agent as a whole. (b) A checklist of at least ten cases, three of them on hostile request text, each naming the wrong behaviour it exists to catch. Run it on the scripted backend — a cap and a gate are your code, and no model can influence whether they fire, so a deterministic run is not the cheap test but the correct one. **And be clear what it does not show:** a scripted case proves the guardrail fires when the agent attempts the bad action, because you scripted the attempt. Whether a live model can be talked into attempting it is a different question, and it is an observation from your D5 battery.

### Governance

- **Plain.** The documented process around the system — who approved it, on what basis, and who can stop it.

**Technically.** Risk classification, approval gates, a model inventory, an audit trail, and a **named accountable owner**. The GOVERN function of the NIST framework in Reading 3 and the accountability dimension of the IMDA frameworks in Reading 4 are both this layer. It runs on a slower clock than the other three, and its blind spot is categorical: **governance sees process, never behaviour** — a fully compliant regime can sit around a model nobody has evaluated on a subgroup. The part that does real work is the last item on the list, and it is the hardest to obtain.

### Human in the loop — stated properly

- **Plain.** A person who has enough time, enough information and the authority to stop it. All three, or it is not oversight.

**Technically.** Specify the **window** (how long between decision and consequence), the **evidence** (what the reviewer sees), and the **authority** (what they may do). Where any is absent the control does not exist regardless of the architecture diagram. **Automation bias** — declining scrutiny after a run of correct outputs — degrades the control over time even where all three are present, which is why Singapore's agentic framework names it explicitly.

### P-U-R-E

- **Plain.** DBS's four-word test for whether a deployed system is fit to face a customer.

**Technically.** Purposeful · Unsurprising · Respectful · Explainable. Introduced in Week 2 as a values statement; read against the four operational layers it is a specification — governance, observability, evaluation-with-slicing, and architecture respectively. Predates large language models and has aged well.

### SOURCE LINKS

NIST AI 600-1 — Generative AI Profile (July 2024), the twelve risk categories · [nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf](https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf)

International AI Safety Report 2026 (3 Feb 2026), chaired by Yoshua Bengio · [internationalaisafetyreport.org/publications](https://internationalaisafetyreport.org/publications)

OWASP Top 10 for LLM Applications 2026 — see Reading 2 · [genai.owasp.org/resource/owasp-genai-llm-top-10-2026/](https://genai.owasp.org/resource/owasp-genai-llm-top-10-2026/)

Anthropic, “Building effective agents” (Dec 2024) — the Week-4 doctrine · [www.anthropic.com/engineering/building-effective-agents](https://www.anthropic.com/engineering/building-effective-agents)

- **Plain.** Week-3 eval harness and Week-4 agent notebooks: your own ``class_3/live-coded-colab/`` and ``class_4/live-coded-colab/``. The Week-1 classifier-vs-API notebook (``class_1/live-coded-colab/PE6201_C2_classifier_vs_api``) is the cleanest example of a silent failure you have already produced with your own hands — the classifier returns a confident label for an input unlike anything it was trained on, and says nothing.

## CAVEATS

1. **A composite assembled by the instructor, not a summary of one source, and the only Class-6 reading the OBT+ outline does not name.** It exists because C1 is a whole capsule on failure modes and none of the assigned governance documents describes a single concrete failure. The two-question diagnostic, the detectability × reversibility grid, the review-budget arithmetic and the four-layer framing are the instructor's; the terminology is NIST's and OWASP's, cited where used.
2. **Every number in the worked examples is illustrative.** The 94% / 96% / 71% subgroup figures and the 50,000-task review budget are plausible placeholders chosen to make the arithmetic legible, not measurements of any real system. **The structure is what transfers; substitute your own numbers.**
3. **The three review signals are priced in orders of magnitude, not in benchmarks.** “Self-consistency triples your tokens” is arithmetic from sampling three times, not a measured cost on any particular task; grounded verification and structural validation have no quoted cost at all because it depends entirely on your sources and your schema.
4. **Generated and fact-checked using Anthropic models** — including on the question of what agents do when they fail, which is a subject the vendor has a position on. The A2 references were checked against the released brief on 5 September 2026. **Flag anything that looks wrong.**